vLLM vs Ollama

Where this fits in our project

Our backend is a multi-tenant RAG chatbot: every company gets its own knowledge base, and chat completions go through a pluggable LlmProvider abstraction that already supports an OpenAI-compatible base_url. A local Ollama instance (qwen2.5:3b-instruct) is registered as one of those providers for offline and demo use, alongside hosted providers like Groq and Cerebras. vLLM also exposes an OpenAI-compatible API, so at the integration level the two are interchangeable. Swapping one for the other later is a provider config change, not a rewrite. The real question is which one to actually run, and when.

The numbers

Two benchmarks put vLLM and Ollama on the same GPU serving the same model, so they're a fair comparison:

Test	Result
Red Hat, A100 GPU, Llama-3.1-8B (source)	vLLM hit 793 tokens/sec at peak vs 41 for Ollama. Response time under load: 80ms (vLLM) vs 673ms (Ollama).
R. McDermott, dual A6000 GPUs, Qwen3-14B (source)	vLLM up to 3.2x faster than Ollama at 128 concurrent requests. Ollama's throughput flattens around ~22 req/sec regardless of added load.

The one spot Ollama wins is a single request with nothing else running at the same time, about 18% faster in the McDermott test.

Why the gap

It comes down to how each one handles multiple requests at once:

- **vLLM** is built around PagedAttention, which manages GPU memory in small chunks instead of one big reserved block per request. That lets it pack far more requests into the same GPU memory and keep pulling new ones in as soon as room frees up. It's built to serve lots of users at once, which matches the shape of our project once more than one company is chatting at the same time.
- **Ollama** (built on llama.cpp) caps how many requests it runs in parallel (4 by default) and doesn't do the same dynamic memory packing. Past that cap, everything else waits. It's built to run a model on one person's own machine, not as a multi-tenant server.

So the gap isn't a fluke or a config problem on Ollama's side. It's what each tool was actually designed to do.

What each one requires

	vLLM	Ollama
OS	Officially Linux only. No native Windows build, needs WSL2 or Docker on Windows.	Native installers for Windows, macOS, and Linux, plus Docker.
GPU	Wants a real GPU with decent VRAM. Its speed advantage only shows up once it can batch a lot of requests in memory at once, so it's not really built for small consumer cards.	Runs fine on small GPUs, falls back to CPU/RAM if needed. Built around quantized models that fit in tight VRAM.
Best fit	A real server with a proper GPU, handling many tenants' requests at once.	One person's own machine, or a quick local demo.

Where this leaves us

This lines up with what's already the working assumption: vLLM is the stronger engine, and the gap is real and structural, not a benchmarking artifact. It comes from a genuine architectural difference (PagedAttention plus continuous batching vs a fixed request cap), not from one project being less polished.

Setup is part of that picture too. vLLM means a Python/pip install or Docker, more moving parts to get right and no native Windows build, while Ollama is one installer and one command to pull a model, running natively on Windows out of the box. Combined with where vLLM's advantage actually shows up (many concurrent requests on a proper GPU) and our current dev box being Windows with a 4GB GTX 1650 serving one person at a time, Ollama is the right fit for our local dev and demo setup right now. vLLM becomes the right call once this project is deployed for real, on a Linux GPU server handling multiple companies at once, and since both speak the same OpenAI-style API, moving over later is just swapping the provider config, not a rewrite.

Sources: Kwon et al., PagedAttention paper (SOSP '23) · Red Hat Developer benchmark · McDermott benchmark · vLLM docs · Ollama repo · llama.cpp repo